Library Management System

Declarative Automation Project Report

Validation Rules & Record-Triggered Flows for Book Issue Management

Prepared By	Ajay Ogirala
Platform	Salesforce Developer Edition
Module	Flow Builder – Record-Triggered Flows & Validation Rules
Project	Library Book Issue Automation
Date	30 July 2026

Table of Contents

1. Project Introduction

This report documents the Day 3 assignment of the Salesforce Interview Readiness Bootcamp, focused on Validation Rules, Record-Triggered Flows, and Apex Triggers. The concepts from the assignment brief were applied to a practical use case – a **Library Management System** – in which a custom **Book Issue** object tracks the issuing of books to members, and a related **Book** object tracks availability status.

The goal of the implementation was to automate the Book Issue process declaratively wherever possible, using Flow Builder and Validation Rules, while reserving Apex for scenarios that genuinely require it. The sections that follow present the automation components in the order they were built and activated in the Salesforce org, supported by screenshots captured directly from Flow Builder and Object Manager.

1.1 Objectives

- Understand the differences between Validation Rules, Flows, and Apex Triggers.
- Select the appropriate automation tool for each business requirement.
- Build simple, reliable Record-Triggered Flows.
- Enforce data quality on the Book Issue object using Validation Rules.
- Be able to explain each design decision confidently in an interview setting.

1.2 Components Implemented

- **Book Issue Date Flow** – automatically sets the Issue Date when a Book Issue record is created.
- **Send Book Issue Email Flow** – sends a confirmation email to the librarian when a book is issued.
- **Book Status Update Flow** – updates the related Book record's status to “Issued.”
- **Validation Rules** – five rules enforcing mandatory fields and correct date logic on Book Issue.

2. Record-Triggered Flow – Book Issue Date Automation

The first automation component addresses the requirement that the Issue Date should be populated automatically whenever a new Book Issue record is created, removing the need for the librarian to enter it manually.

2.1 Configuring the Flow Trigger

A new Record-Triggered Flow named **Book Issue Date Flow** was created on the **Book Issue** object. The trigger condition was set to *"A record is created,"* with no entry conditions, so the flow runs for every new Book Issue record. The flow was optimized for **Fast Field Updates**, meaning it runs before the record is saved to the database — the most efficient option when only fields on the triggering record need to be changed.

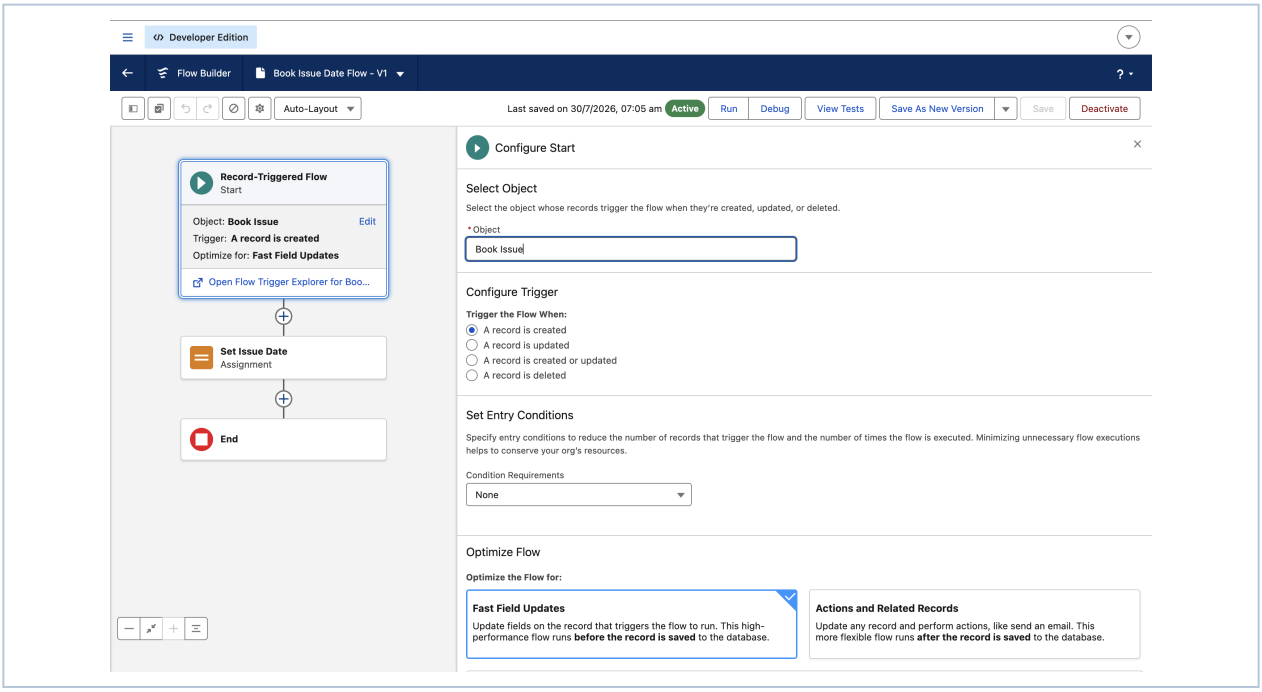

Figure 1: Configuring the Start element of the Book Issue Date Flow – object, trigger type, and optimization set to Fast Field Updates.

2.2 Assigning the Issue Date

An Assignment element named **Set Issue Date** was added immediately after the Start element. It sets the **Issue Date** field on the triggering Book Issue record equal to **CurrentDate**, sourced from the flow's system information. Because the flow runs before the record is saved, this update does not require a separate DML operation, keeping the automation lightweight.

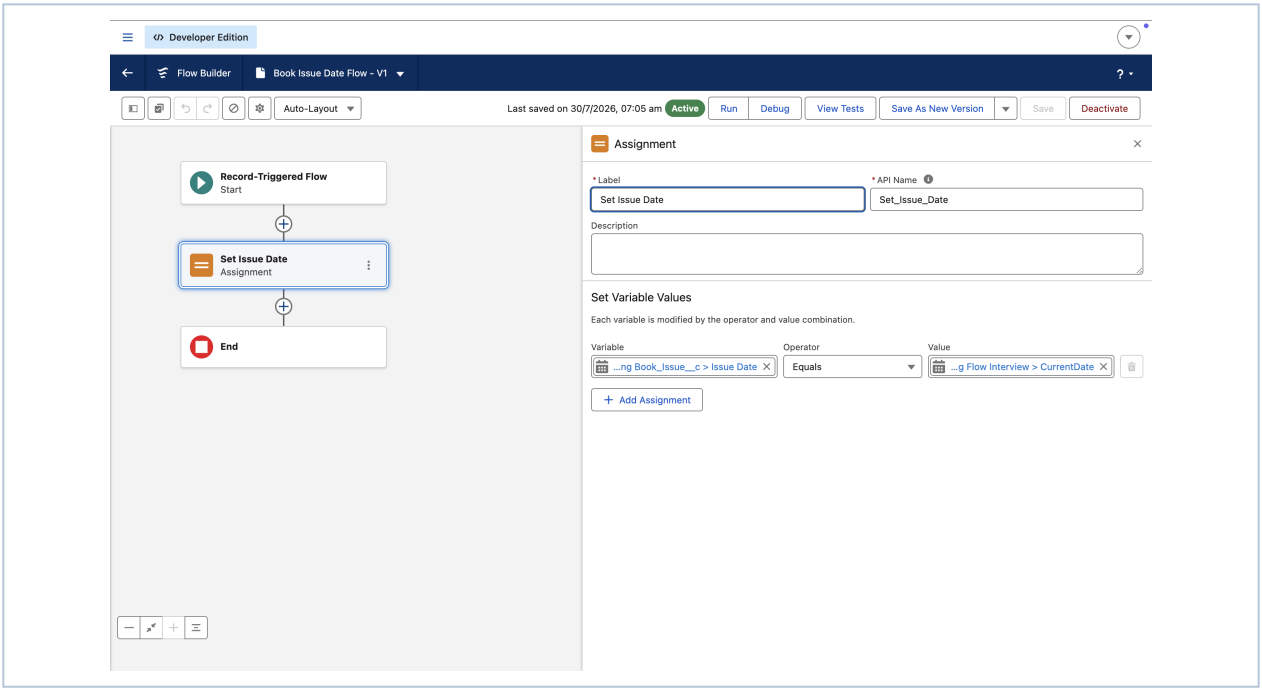

Figure 2: The Assignment element “Set Issue Date,” mapping Book Issue > Issue Date to the flow's CurrentDate value.

3. Record-Triggered Flow – Book Issue Email Notification

The second requirement was to notify the librarian by email whenever a book is issued. A separate flow, **Send Book Issue Email**, was created with a **Send Email** action running after the record is saved, since sending an email is a side effect rather than a field update.

3.1 Initial Configuration and Error Encountered

During the first build, the Send Email action referenced $Record.Student__c to personalize the recipient, which does not exist on the Book Issue object (the correct relationship field is **Member**, not Student). Flow Builder's Errors and Warnings panel correctly blocked activation, reporting an invalid reference and leaving the flow in an **Inactive** state until the issue was resolved.

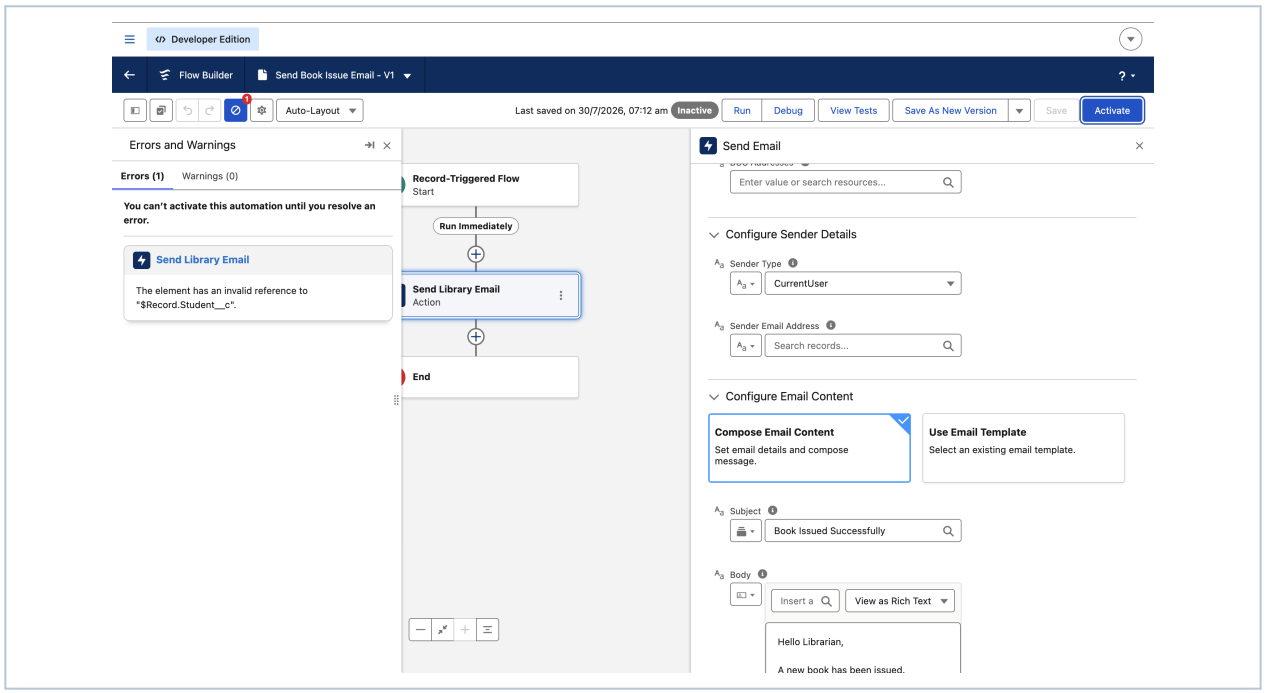

Figure 3: Flow Builder blocking activation of Send Book Issue Email due to an invalid reference to $Record.Student__c.

3.2 Resolving the Error and Activating the Flow

The faulty reference was removed and the Send Email action was reconfigured with a static recipient address (**librarian@college.com**), a fixed subject line (“Book Issued Successfully”), and a composed email body confirming the issue. After the fix, the Errors and Warnings panel reported **zero errors**, and the flow was successfully activated.

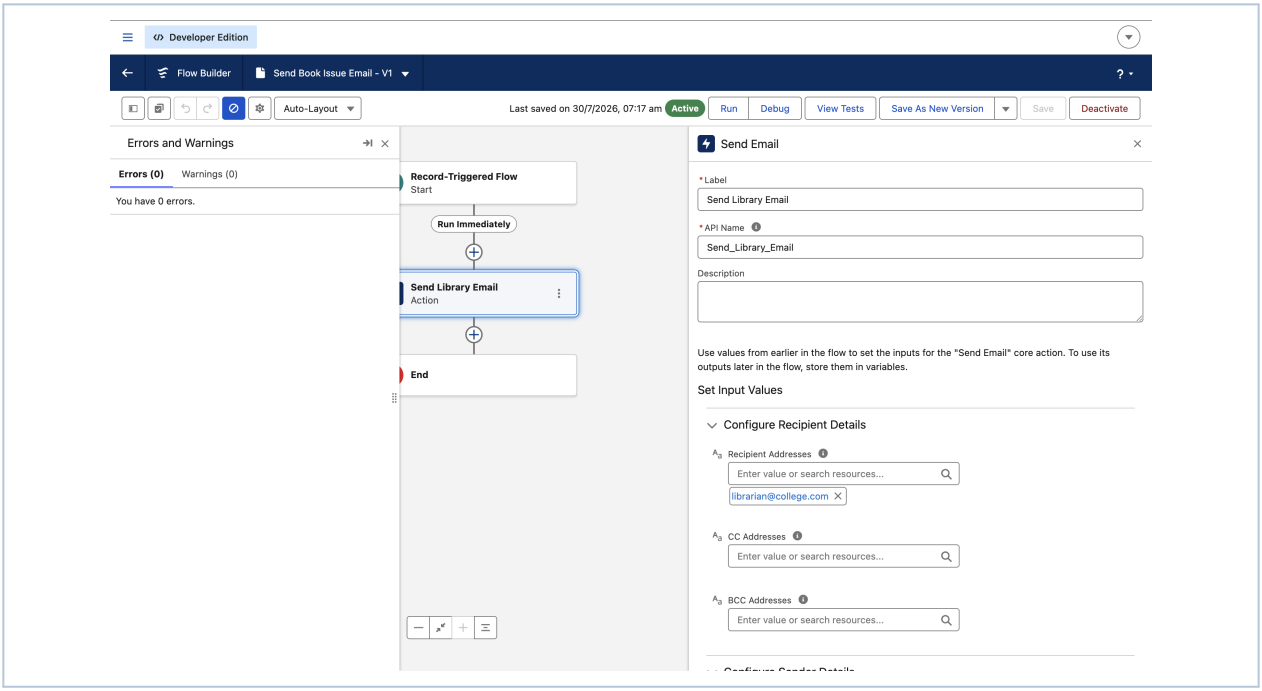

Figure 4: Send Book Issue Email flow after debugging – zero errors reported and the flow set to Active with recipient configured.

4. Record-Triggered Flow – Book Status Update

To keep book availability accurate, a third flow, **Book Status Update Flow**, updates the related **Book** record whenever a Book Issue record is created. This flow uses an **Update Records** element configured with the option “Specify conditions to identify records, and set fields individually,” since the record to update (the Book) is different from the record that triggered the flow (the Book Issue).

The Update Records element filters the **Book** object where **Record ID** equals the Book looked up on the triggering Book Issue record, and sets the **Status** field to **“Issued.”** This keeps the Book's availability in sync automatically, without requiring the librarian to update it by hand.

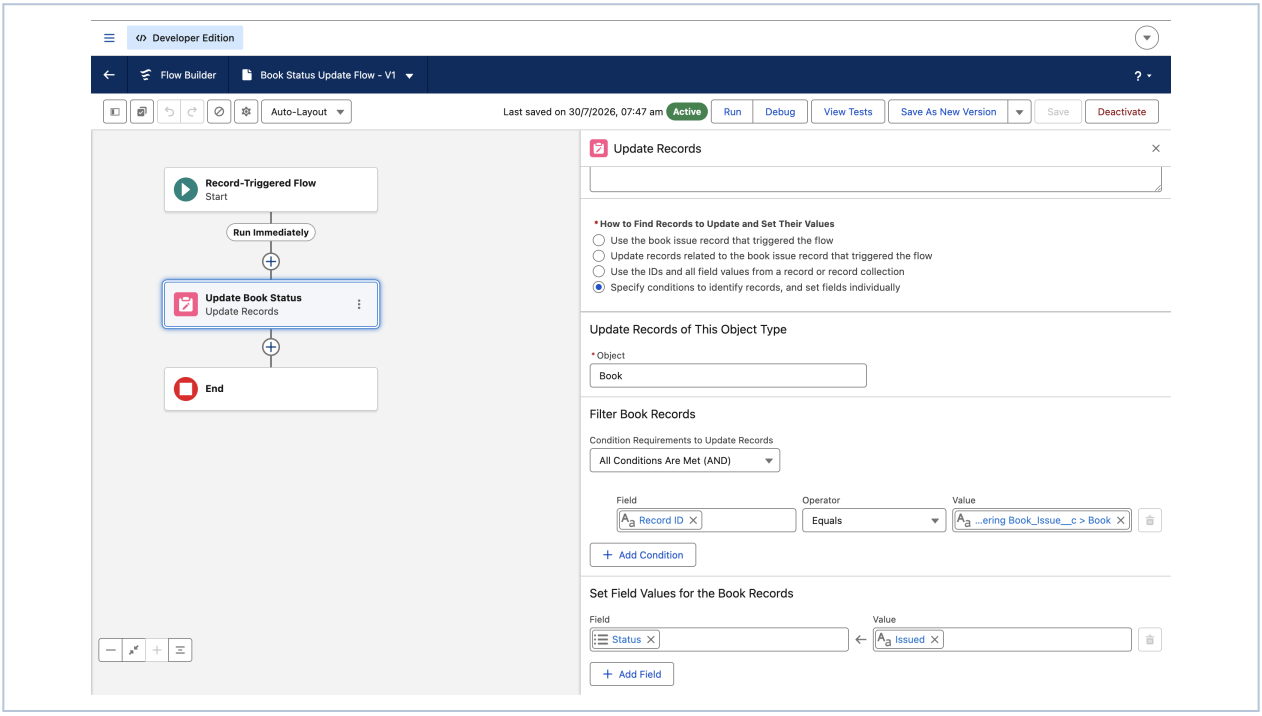

Figure 5: Update Records element of the Book Status Update Flow – filtering the related Book by Record ID and setting Status to “Issued.”

5. Validation Rules – Data Quality Enforcement

Alongside the flows, five Validation Rules were created on the Book Issue object to enforce data integrity at the point of entry. Validation Rules were chosen over Flow or Apex for these checks because they run synchronously before save, block invalid data unconditionally, and require no code — making them the simplest and most reliable tool for this class of requirement.

5.1 Check_Return_Date

This rule guarantees that a book cannot be returned before it was issued, by comparing the Issue Date and Return Date fields directly.

```
Issue_Date__c > Return_Date__c
```

Error Message: “Return Date must be after Issue Date.” | **Error Location:** Top of Page

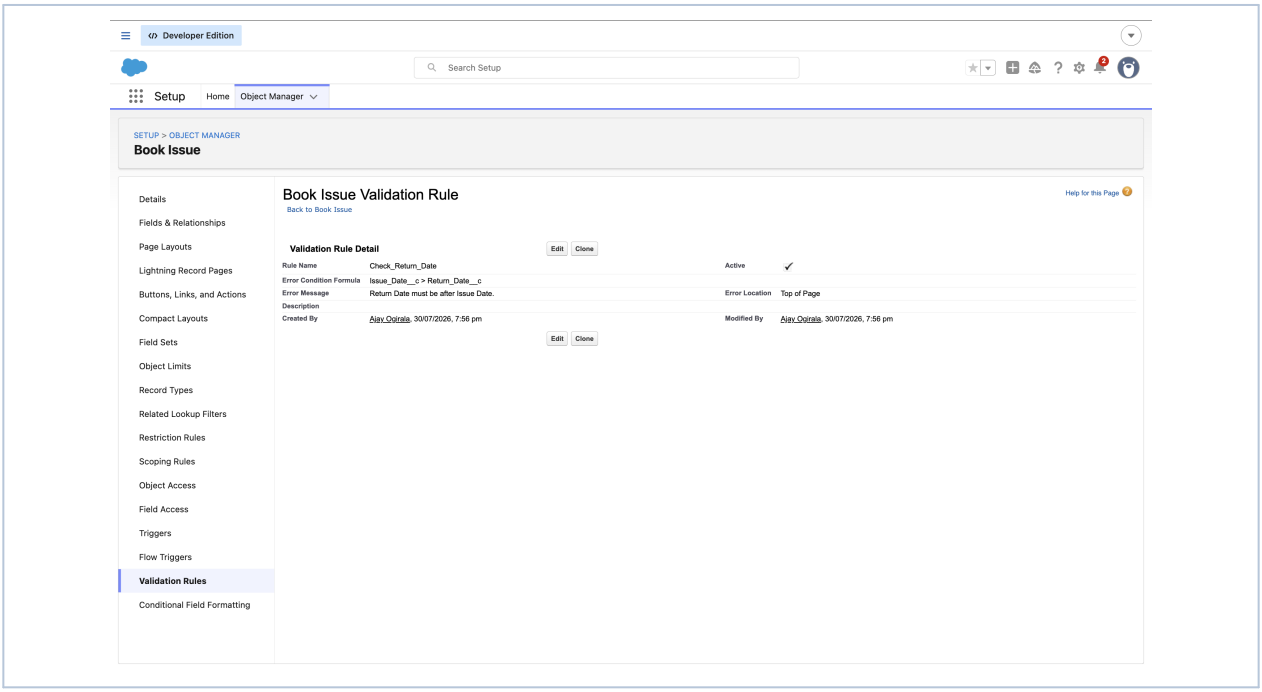

Figure 6: Detail view of the Check_Return_Date validation rule, comparing Issue Date and Return Date.

5.2 Member_Required

This rule enforces that every Book Issue record must have a Member associated with it, preventing incomplete records from being saved.

```
ISBLANK(Member__c)
```

Error Message: “Please select a Member.” | **Error Location:** Top of Page

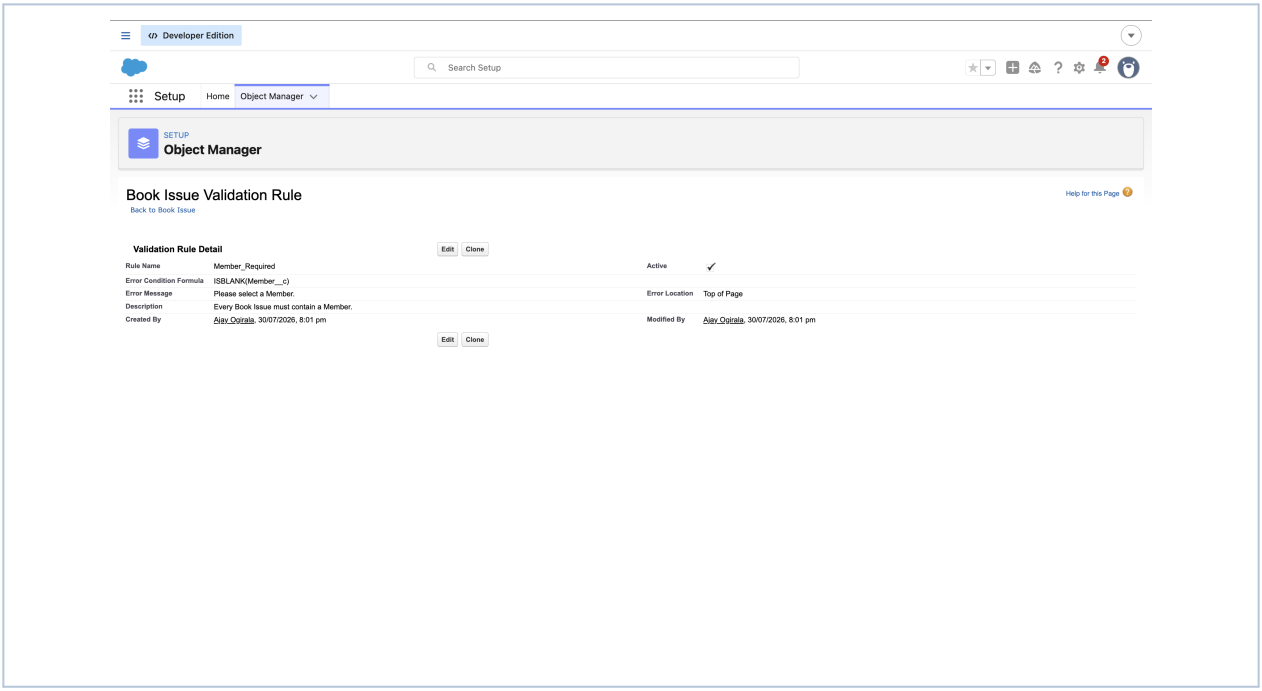

Figure 7: Detail view of the Member_Required validation rule, blocking save when Member is blank.

5.3 Complete Set of Validation Rules

In addition to the two rules detailed above, three further rules were implemented to cover book selection, availability, and return-date consistency. The complete list, as it appears in Object Manager, is shown below.

Rule Name	Error Location	Error Message
Book_Available	Top of Page	This book has already been issued.
Book_Required	Top of Page	Please select a Book.
Check_Return_Date	Top of Page	Return Date must be after Issue Date.
Invalid_Return_Date	Return Date	Return Date cannot be earlier than Issue Date.
Member_Required	Top of Page	Please select a Member.

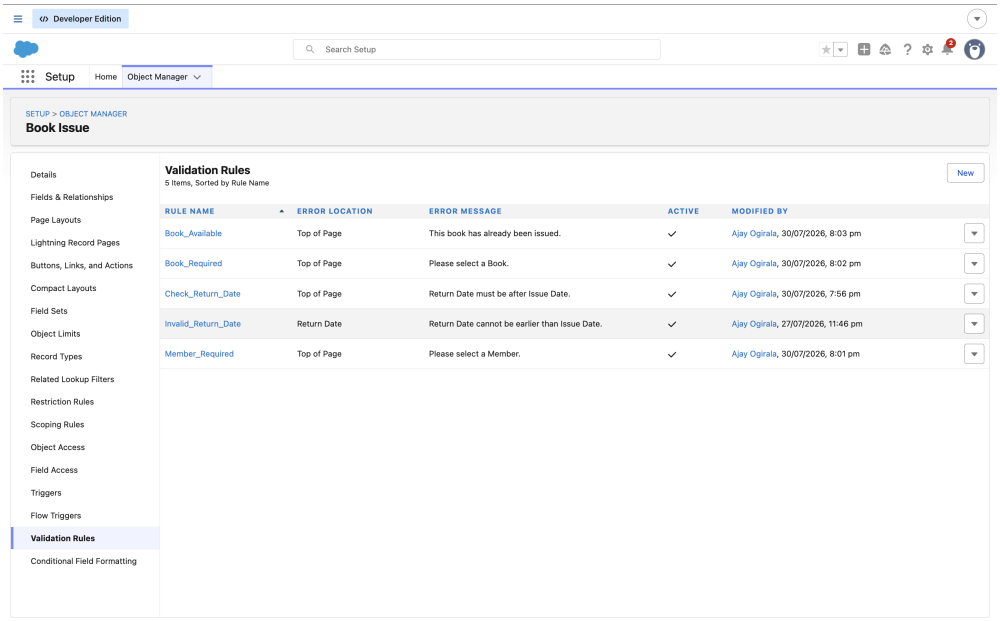

Figure 8: Object Manager – Validation Rules list for Book Issue, showing all five active rules.

6. Summary and Design Rationale

Every requirement in this project was met using declarative tools, in line with the guiding principle that a good Salesforce developer writes less Apex whenever point-and-click automation can solve the problem.

Requirement	Tool Used	Reasoning
Auto-fill Issue Date	Flow (Fast Field Updates)	Simple same-record field update before save; no DML needed.
Send confirmation email	Flow (Actions after save)	Sending email is a side effect, best run after the record commits
Update related Book status	Flow (Update Records)	Requires updating a different record than the trigger — Flow ha
Mandatory fields (Member, Book)	Validation Rule	Simple, synchronous, no-code enforcement.
Return Date after Issue Date	Validation Rule	Pure field comparison, no DML required.
Prevent duplicate/unavailable book issue	Validation Rule	Blocks save unconditionally with minimal overhead.

No Apex was required for this scope of work. Apex Triggers would become necessary if the system needed to call an external REST API, perform complex multi-object calculations, or process very large data volumes (for example, a bulk import of thousands of Book Issue records) with logic beyond what Flow can efficiently express.